Stage One Follow-up Package: R20.2, the LKR 298,375.00 Gap and Six Routes

Oct 14, 2026 · @Shantha

This package proposes the next read-only checks and the analysis you asked for. Nothing in it has been run, and the investigation is not complete.

- **Authorises:** preparation and analysis only. No database statement has been sent since the Stage One report.
- **Target, when approved:** your localhost application and schema `avenir` only.
- **Sources:** the Stage One results, the code copy, and the three target files staged on 14 October. The code copy is identical to the target for every file used here except TourPaymentDeskController.php, Bulk.php and Vouchers.php. For those three, the staged target files are used.
- **Unchanged:** the Stage One report, A1, A2, r3.3–r3.5 and the Security proposal.

Labels: **[Observed]** means read in the code or in Stage One results. **[Deduced]** means reasoning from those reads. **[Assumption]** means not checked. **[Proposal]** means for your decision.

1. Tracing the LKR 298,375.00 difference

Six SELECTs would trace the difference: two metadata reads, the section 8 comparison, a per-voucher reconciliation, and two detail reads. None of them assumes a cause. The fact that voucher 7134's amount equals the difference is treated only as a coincidence to test. **[Proposal]**

What is known. [Observed]

- R0's 6,833,545.97 is the sum of credits to 1-6-8-9750 under the numbers of the 38 sup = 0 vouchers. R23.1's 6,535,170.97 is the sum of those vouchers' own Amount.
- Authorisation (VoucherPaymentController::update, status 1) reads the voucher's **lines**: v_inv rows whose V_tkn equals the voucher number. It posts one journal per line, for the line's Amount, Dr the line's account and Cr 1-6-8-9750 when sup is not 1. It does not use voucher.Amount, and it does not check that the voucher still awaits authorisation.

So the ledger credit equals the sum of the lines at the moment of authorisation, once for every authorisation call. **[Deduced]**

Explanations to test. None is assumed.

#	Possible cause	What would show it
E1	A voucher's lines total more than its Amount (a line added later, an orphan line under its number, or the Amount edited after authorisation)	line_total ≠ voucher_amount in G2
E2	A voucher was authorised more than once. The code allows it; whether it happened is not known	journals_9750 greater than lines in G2; repeated legs in G3
E3	Rows from another source carry the same number in Recpt_No, which also holds receipt numbers	cr_9750 ≠ its "voucher payment" part in G2; other description types in G3
E4	Two voucher rows share the number, so a join counts the ledger rows twice. R0's own summing method is not on record	voucher_rows > 1 in G2; M2 shows whether voucher.Vnum has a unique index
E5	A combination of the above, spread over several vouchers	G2 differences that add up to 298,375.00 across more than one voucher

M1. The columns these reads use, with types and defaults. v_inv was not in T3.

```
SELECT /*+ MAX_EXECUTION_TIME(60000) */ c.TABLE_NAME, c.COLUMN_NAME,
c.DATA_TYPE, c.COLUMN_TYPE, c.IS_NULLABLE, c.COLUMN_DEFAULT
FROM information_schema.COLUMNS c
WHERE c.TABLE_SCHEMA = 'avenir'
  AND ((c.TABLE_NAME = 'v_inv' AND c.COLUMN_NAME IN ('ID', 'V_tkn',
'Invoice', 'account', 'Amount', 'RCType', 'Date'))
    OR (c.TABLE_NAME IN ('account_runing', 'account_runing_2024') AND
c.COLUMN_NAME IN ('aflag', 'Bank_Date', 'Recpt_No', 'Date', 'Discription',
'Dr', 'Cr', 'ID', 'L_Code'))
    OR (c.TABLE_NAME = 'voucher' AND c.COLUMN_NAME IN ('Vnum', 'Amount',
'W_Date', 'Aflag', 'sup')))
ORDER BY c.TABLE_NAME, c.COLUMN_NAME;
```

M2. Table sizes, engines and indexes. Section 2 uses this as well.

```
SELECT /*+ MAX_EXECUTION_TIME(60000) */ t.TABLE_NAME, t.ENGINE, t.TABLE_ROWS,
s.INDEX_NAME, s.NON_UNIQUE, s.SEQ_IN_INDEX, s.COLUMN_NAME
FROM information_schema.TABLES t
LEFT JOIN information_schema.STATISTICS s ON s.TABLE_SCHEMA = t.TABLE_SCHEMA
AND s.TABLE_NAME = t.TABLE_NAME
WHERE t.TABLE_SCHEMA = 'avenir'
  AND t.TABLE_NAME IN ('voucher', 'v_inv', 'account_runing',
'account_runing_2024', 'user_acc_tbl')
ORDER BY t.TABLE_NAME, s.INDEX_NAME, s.SEQ_IN_INDEX;
```

G1. The section 8 comparison, exactly as written there.

```
SELECT v.Vnum AS voucher_no, v.Amount AS voucher_amount,
       SUM(a.Cr) AS credits_to_9750, COUNT(DISTINCT a.ID) AS journals
FROM avenir.voucher v
JOIN avenir.account_runing a ON a.Recpt_No = v.Vnum AND a.L_Code = '1-6-8-
9750'
WHERE v.sup = 0 AND v.Aflag = 1
GROUP BY v.Vnum, v.Amount
HAVING SUM(a.Cr) <> v.Amount
ORDER BY v.Vnum;
```

G1 has five limits:

- It joins ledger rows to every voucher row with that number, so a shared number (E4) would double its sums.
- It reads only account_runing and only credits.
- It drops vouchers with no 9750 rows (inner join).
- It does not separate authorisation rows from other rows (E3).
- It lists only vouchers that differ, so the matching ones cannot be seen.

G1 is therefore a first look. G2 is the reconciliation.

G2. Per-voucher reconciliation of all 38 vouchers. Every source is summed on its own before the join, so no row can be counted twice.

```
SELECT /*+ MAX_EXECUTION_TIME(60000) */
       v.Vnum AS voucher_no, v.ID AS voucher_id, DATE(v.W_Date) AS
voucher_date, v.Amount AS voucher_amount,
       vn.voucher_rows,
       COALESCE(li.line_count, 0) AS line_count, COALESCE(li.line_total, 0)
AS line_total,
       COALESCE(c.rows_9750, 0) AS rows_9750, COALESCE(c.journals_9750, 0) AS
journals_9750,
       COALESCE(c.cr_9750, 0) AS cr_9750, COALESCE(c.dr_9750, 0) AS dr_9750,
       COALESCE(c.vp_cr_9750, 0) AS cr_9750_voucher_payment_rows,
       COALESCE(o.rows_9750_2024, 0) AS rows_9750_in_2024_table
FROM avenir.voucher v
JOIN (SELECT w.Vnum, COUNT(*) AS voucher_rows FROM avenir.voucher w GROUP BY
w.Vnum) vn ON vn.Vnum = v.Vnum
LEFT JOIN (SELECT i.V_tkn, COUNT(*) AS line_count, SUM(i.Amount) AS
line_total FROM avenir.v_inv i GROUP BY i.V_tkn) li ON li.V_tkn = v.Vnum
LEFT JOIN (SELECT a.Recpt_No, COUNT(*) AS rows_9750, COUNT(DISTINCT a.ID) AS
journals_9750, SUM(a.Cr) AS cr_9750, SUM(a.Dr) AS dr_9750,
                  SUM(CASE WHEN UPPER(a.Discription) LIKE 'VOUCHER PAYMENT%'
THEN a.Cr ELSE 0 END) AS vp_cr_9750
           FROM avenir.account_runing a WHERE a.L_Code = '1-6-8-9750' GROUP
BY a.Recpt_No) c ON c.Recpt_No = v.Vnum
LEFT JOIN (SELECT b.Recpt_No, COUNT(*) AS rows_9750_2024 FROM
avenir.account_runing_2024 b WHERE b.L_Code = '1-6-8-9750' GROUP BY
b.Recpt_No) o ON o.Recpt_No = v.Vnum
WHERE v.sup = 0 AND v.Aflag = 1
ORDER BY v.Vnum;
```

Expected: 38 rows if every number is unique. The checks are:

- SUM(voucher_amount) should be 6,535,170.97 and SUM(cr_9750) should be 6,833,545.97.
- A different cr_9750 total means R0′s figure was not reproduced. That is reported, not reconciled.

G3. Every leg of every journal that credits 1-6-8-9750 under the 38 numbers.
Descriptions are reduced to the same categories as R21.

```
SELECT /*+ MAX_EXECUTION_TIME(60000) */
       a.Recpt_No AS voucher_no, a.ID AS journal_id, a.L_Code, a.Dr, a.Cr,
a.Date, a.Dt AS entered_at,
       CASE WHEN UPPER(a.Discription) LIKE 'VOUCHER PAYMENT%' THEN 'voucher
payment'
            WHEN a.Discription LIKE 'Hotel Reservation%' THEN 'hotel
reservation accrual'
            WHEN a.Discription LIKE 'Lunch or Other Fee%' THEN 'lunch or
other fee'
            WHEN a.Discription LIKE 'Brought Forward Balance%' THEN 'brought
forward'
            WHEN a.Discription IS NULL OR a.Discription = '' THEN '(empty)'
            ELSE 'other (text withheld)' END AS description_type,
       a.Invoice_Num AS line_invoice, a.aflag, a.Bank_Date,
       (SELECT GROUP_CONCAT(u.id ORDER BY u.id) FROM avenir.user_acc_tbl u
WHERE u.user = a.Cashier) AS entering_login_ids
FROM avenir.account_runing a
WHERE a.ID IN (SELECT b.ID FROM avenir.account_runing b
               WHERE b.L_Code = '1-6-8-9750'
                 AND b.Recpt_No IN (SELECT v.Vnum FROM avenir.voucher v WHERE
v.sup = 0 AND v.Aflag = 1))
ORDER BY a.Recpt_No, a.ID, a.Dr DESC, a.L_Code;
```

G4. The lines of the 38 vouchers. Description and Payee are left out because they can hold names.

```
SELECT /*+ MAX_EXECUTION_TIME(60000) */ i.V_tkn AS voucher_no, i.ID AS
line_id, i.Invoice AS line_invoice, i.account, i.Amount, i.RCType, i.Date
FROM avenir.v_inv i
WHERE i.V_tkn IN (SELECT v.Vnum FROM avenir.voucher v WHERE v.sup = 0 AND
v.Aflag = 1)
ORDER BY i.V_tkn, i.ID;
```

Scope and limits.

- Scope: schema `avenir` only, using the tables voucher, v_inv, account_runing, account_runing_2024 and user_acc_tbl. Only ids are read from user_acc_tbl.
- Size: R0 counted 125 rows on 1-6-8-9750 in all, so G3 should return well under 300 rows. Each statement is capped at 60 seconds.
- History: the reads show the present state only. Nothing records which screen or call wrote a row. Legacy screens keep no audit of edits to voucher amounts or lines (tpd_audit covers desk actions only, from 11 October). Rows deleted earlier cannot be seen, except any copied to account_runing_del, whose newest row R0 found dated 19 Sep 2025.

- Journal numbers: these can collide between screens (SX-7). If an ID is shared, G3 also returns legs posted under other numbers, which show up because of their different voucher_no.
- V_tkn: if it is a text column (M1 shows), the join with Vnum converts types. A stored value with spaces would not match and would show as a missing line.
- Result: the reads can show where the extra LKR 298,375.00 sits. They cannot show why. Any correction is the accountant's decision.

2. R20.2 revised

One replacement statement, R20.2r, gives R20.2's answer with the same coverage, and adds which ledger table each candidate is in. It avoids R20.2's repeated per-voucher scans. A 2-minute cap ends it, and releases its read locks, if it runs long. **[Proposal]** The original 36-minute statement is not proposed again.

What R20.2 asked. R20.2 takes each authorised voucher found by number in neither ledger table: the 3,921 that R20.1 implies. It counts how many have at least one "VOUCHER PAYMENT" row with the same date and with Dr or Cr equal to the voucher amount, in either table, grouped by voucher month.

Why it took 36 minutes. [Deduced] The query plan was not read.

- For each of the 3,921 vouchers, the OR of two correlated EXISTS can mean a full pass over both ledger tables, with UPPER() worked out on every row.
- R0 counted 11,364 and 15,900 rows in those tables. That is up to about 107 million row tests.
- The by-number part of the same filter took 0.39 seconds in R20.1, so the time was spent on the date-and-amount part.

R20.2r.

```
SELECT /*+ MAX_EXECUTION_TIME(120000) */
       DATE_FORMAT(v.W_Date, '%Y-%m') AS voucher_month,
       COUNT(*) AS not_found_by_number,
       SUM(cur.d IS NOT NULL OR old.d IS NOT NULL) AS
candidate_same_date_and_amount,
       SUM(cur.d IS NOT NULL AND old.d IS NULL) AS
candidate_current_table_only,
       SUM(cur.d IS NULL AND old.d IS NOT NULL) AS candidate_2024_table_only,
       SUM(cur.d IS NOT NULL AND old.d IS NOT NULL) AS candidate_both_tables
FROM avenir.voucher v
LEFT JOIN (SELECT a.Date AS d, a.Dr AS amt FROM avenir.account_runing a WHERE
UPPER(a.Discription) LIKE 'VOUCHER PAYMENT%'
           UNION
```

```
            SELECT a.Date, a.Cr FROM avenir.account_runing a WHERE
UPPER(a.Discription) LIKE 'VOUCHER PAYMENT%') cur
       ON cur.d = DATE(v.W_Date) AND cur.amt = v.Amount
LEFT JOIN (SELECT b.Date AS d, b.Dr AS amt FROM avenir.account_runing_2024 b
WHERE UPPER(b.Discription) LIKE 'VOUCHER PAYMENT%'
            UNION
            SELECT b.Date, b.Cr FROM avenir.account_runing_2024 b WHERE
UPPER(b.Discription) LIKE 'VOUCHER PAYMENT%') old
       ON old.d = DATE(v.W_Date) AND old.amt = v.Amount
WHERE v.Aflag = 1
  AND NOT EXISTS (SELECT 1 FROM avenir.account_runing a WHERE a.Recpt_No =
v.Vnum AND UPPER(a.Discription) LIKE 'VOUCHER PAYMENT%')
  AND NOT EXISTS (SELECT 1 FROM avenir.account_runing_2024 b WHERE b.Recpt_No
= v.Vnum AND UPPER(b.Discription) LIKE 'VOUCHER PAYMENT%')
GROUP BY voucher_month
ORDER BY voucher_month;
```

Same coverage. [Deduced from the statement texts]

- The WHERE clause is R20.2′s word for word, so the same vouchers are counted.
- A ledger row matches a voucher in R20.2 when its date equals the voucher date and its Dr or its Cr equals the amount. That is true exactly when the pair (date, Dr) or the pair (date, Cr) equals (voucher date, amount).
- Each derived table holds every such pair from all “VOUCHER PAYMENT” rows of one ledger table, with the same description test. Both tables and both sides are covered.
- Rows with a NULL date or amount match nothing, in R20.2 and here alike. Zero amounts are kept, as in R20.2.

No double counting. UNION, rather than UNION ALL, keeps each (date, amount) pair once in each derived table. The join is on that pair, so a voucher meets at most one row from each table. Each voucher therefore appears exactly once, and COUNT(*) counts vouchers, as R20.2 did.

Two checks are built into the result:

- not_found_by_number for each month must equal R20.1′s not_in_current_ledger minus found_by_number_in_2024: 3,921 in all. Any difference stops the check and is reported.
- The three “only or both” columns must add up to candidate_same_date_and_amount for each month.

Shorter read locks. voucher and account_runing are MyISAM, so a SELECT holds a read lock on them for the whole statement. M2 shows account_runing_2024′s engine.

- **No repeated scans.** Each ledger table is read twice to build the two pair sets, about 54,500 row reads, instead of up to one pass per voucher.

- **Time cap.** `MAX_EXECUTION_TIME(120000)` stops the SELECT after 2 minutes. MySQL then returns error 3024 and releases the locks. The run would be reported as failed, not retried.
- **Your timing.** You choose a time when the local app is idle.

Expected duration: [Not established]. Stage One's timings on the same tables were 0.21–0.69 seconds for one full pass (R21.1, R20.3). Seconds rather than minutes is therefore likely, but it is not claimed. The optional plan read below gives better evidence before anything runs.

Optional before R20.2r: E1, the query plan. This needs your separate approval because it is not a SELECT.

- It is `EXPLAIN FORMAT=TREE` followed by the R20.2r text. EXPLAIN without ANALYZE does not run the query.
- It shows whether MySQL builds each pair set once and looks vouchers up by key.
- If the plan shows a pass over a ledger table for each voucher, R20.2r is not run, and I report back.

Limits.

- A candidate is a row with the same date and amount, not proof that it is that voucher's journal. Another voucher's journal can match too, so the counts are upper bounds, as A2 said for R20.2.
- The hint applies only to a top-level SELECT in MySQL 8.4. If MySQL does not honour it, the capture will show a warning, and that is reported.
- The by-month split is kept only because R20.2 used it. No claim is made that splitting by year or month is faster.

3. Containment for the six additional routes

All six can be contained. Two are best refused (delete_advance and hotel_payment) and four need guards. Those four would share the A2 locks, plus one more: the voucher-number lock the desk already uses. **[Proposal]** The code basis follows. Files are the code copy where its checksum equals the target's (F11), and the staged target files for the desk and its libraries.

Route (POST)	Writes	Main risk	Proposed safeguard	Effect on staff
R1 dashboard2/transport/driver_advance/delete_advance	deletes a pending voucher and its lines	deleting a voucher that is being authorised	**refuse**; reject through Bank Payment Check instead (alternative: guard)	transport staff ask Accounts to reject a wrong advance
R2 tour/reserve_hotel/hotel_payment	night flags, then a line, then a voucher	double payment of the same nights; flags left at 10 with no voucher	**refuse**; hotel advances go through payment requests (alternative: guard)	reservations staff raise a request instead of paying from Booking Details
R3 tour/tour_accounts/accept_approved_expenses	journals per expense; a settlement voucher per driver	journals posted twice; voucher without lines	**guard** (tour lock, claim each expense)	a second click is refused
R4 tour/tour_payment_dashboard/bulk_create	vouchers, lines and batch records	voucher for a request cancelled meanwhile; incomplete undo	**guard** (request locks, status re-read)	“try again” when a request is in use
R5 tour/tour_payment_dashboard/initiate	initiation record; voucher bank and amount	bank change silently skipped after authorisation	**guard** (AUTH lock)	may wait up to 10 seconds
R6 tour/tour_payment_dashboard/bulk_initiate	R5 once per voucher	as R5, for a whole batch	**guard** (AUTH held for the run)	other authorisations wait during a bulk initiation

Common to all six. [Observed] unless marked.

- **X1. Voucher numbers come from two kinds of writer.**
  - Vouchers::issue (R4) takes the named lock `tpd_voucher_issue` and checks that the numbers are free before writing.
  - InvoiceNumberModel::incrementVoucherToken, used by R2, R3 and older screens, reads the counter and writes it back with no lock.
  - So two writers can take the same number **[Deduced]**.
  - Setup adds a unique index on voucher.Vnum only if no duplicates existed. Whether the target has it is not established; M2 shows it.
- **X2. Authorising a voucher that has no lines posts nothing but still marks it authorised.** VoucherPaymentController::update loops over the lines, then sets Aflag 1 regardless.
- **X3. Lines are tied to a voucher by number only (v_inv.V_tkn).** A line left without its voucher is posted with whichever voucher holds that number when it is authorised **[Deduced]**. This is one possible route to E1 in section 1. It is not assumed to have

happened.

- **X4. None of the six is in the E8 route registry**, which covers accounts/* only. The global CSRF filter is commented out in Filters.php. Only the sign-in filter applies to all six.
- **X5. A2′s AUTH and REQ locks bind only code that asks for them.** These six would have to be added to C0.15 and C0.8.

R1. delete_advance

- **Reached from:** transport_advance.php:784. **Internal calls:** none. **Method:** DriverAdvancePaymentController :383.
- **Existing checks:** a voucher with that number exists; that row′s Aflag is 0; the tour is not cancelled.
- **Writes, in order:** deletes every voucher row with that number; deletes every line with that number; sets the tour′s transport_advance marker to 0.
- **Conflicts [Deduced]:**
  - Nothing stops Bank Payment Check (V1) from authorising between the Aflag check and the delete. The journal would stay with no voucher.
  - Only the first row′s Aflag is checked, but all rows under the number are deleted (X1).
  - The tour′s marker is reset even if another advance voucher remains.
  - Whether the page can offer a voucher raised from a payment request is not established. If it can, the request keeps voucher_created = 1 with no voucher.
- **Partial failure:** voucher deleted but lines kept leaves orphan lines (X3). If the marker update fails, the marker stays at 1.
- **Safeguard:**
  - **Refuse** (recommended). A wrong pending advance is rejected through Bank Payment Check, with a remark, which keeps the record.
  - Alternative guard: take AUTH, re-read every row under the number, and refuse if more than one voucher row, any Aflag other than 0, any ledger row or any linked request exists. Reset the marker only when no other advance remains.
- **Staff:** transport staff ask Accounts to reject instead of deleting. Rejection does not reset the marker [Observed: the reject branch does nothing for transport]. Whether that blocks a new advance needs a read (F-3 below).

R2. hotel_payment

- **Reached from:** booking_details.php:2477; the forecast page′s call is commented out. **Internal calls:** none. **Method:** HotelManagementController :480.

- **Existing checks:**
  - the tour is not cancelled, and hotel, amount, reference and bank are filled in;
  - the amount is a number of 0 or more;
  - every night is confirmed (flag 1);
  - it refuses only if a night already has hotel_payment_flag 1 and its paid dollars reach the total.
- **Writes, in order:**
  1. Each night gets flag 10 and its paid dollars increased.
  2. The voucher counter is taken, with no lock (X1).
  3. The line is written **before** the voucher.
  4. The voucher is written: sup 1, Aflag 0, advances_pay 2.
  - Later, V1 posts Dr hotel / Cr bank and sets the nights to 1. Rejection sets them to 2 and takes the dollars back.
- **Conflicts [Deduced]:**
  - Flag 10 is not checked, so a second click or a second person creates a second voucher and adds the dollars again.
  - Payment requests for the same tour and hotel are not checked, so the same nights can be paid by request and by this route.
  - Flag 10 blocks booking cancellation (B1).
- **Partial failure:** if the voucher write fails, the nights stay at 10 with inflated dollars and an orphan line, and no voucher exists to reject. The nights cannot be released or cancelled in the system.
- **Safeguard:**
  - **Refuse** (recommended, consistent with A2′s refusal of save_payment).
  - Alternative guard: a lock per tour and hotel plus `tpd_voucher_issue`. Refuse if any night is at 10 or an open request exists for that tour and hotel. Write the voucher first and check every write.
- **Staff:** reservations staff stop recording hotel advances from Booking Details and raise a payment request for Accounts to approve.

R3. accept_approved_expenses

- **Reached from:** approved_expenses_report.php:521; payment_approval_v2.php:1476. **Internal calls:** none. **Method:** TourAccountsController :2239.
- **Existing checks:**
  - the r1b approver role, a tour and a bank account;
  - at least one expense “approved”;

  - log-sheet mileage for every allocated driver;
  - no settlement voucher already raised for that tour and driver;
  - a payable amount above 0.
  - There is no own-request check in this method.
- **Writes, in order:**
  - For each expense: a journal number, two ledger legs (Dr supplier / Cr driver, no Recpt_No), then the expense is marked “posted”.
  - For each driver: a voucher number with no lock (X1), a settlement voucher (sup 1, Aflag 0, advances_pay 11), then its line.
- **Conflicts [Deduced]:**
  - Two calls both read the same “approved” expenses before either marks them, so every journal is posted twice and two settlement vouchers can be raised.
  - A rejection of the same expenses can run at the same time.
  - Advances authorised (V5, V1) or deleted (R1) after the amount is worked out make the settlement wrong.
- **Partial failure:** a journal posted but its expense not marked is posted again on retry. A voucher written without its line would be authorised with no journal (X2).
- **Safeguard: guard.** Refusal would stop driver settlements.
  - Lock the tour.
  - Claim each expense with a conditional status change before posting its journal, and post only those claimed.
  - Check for an existing settlement under the same lock.
  - Take `tpd_voucher_issue` for the number, and check every write.
  - An own-request check is a separation decision for you.
- **Staff:** none in normal use; a second click is refused.

R4. bulk_create

- **Reached from:** tpd_payables.php:328. **Internal calls:** no controllers. It runs Bulk::create, Batches, Vouchers::issue and SupplierBank::verify. **Method:** desk :146 (target file).
- **Existing checks:**
  - the batch privilege, Setup version 7 or later, and a paying account;
  - a verified supplier bank, or confirmation by a verifier;
  - the separation check;
  - a one-time submit token;
  - eligibility: status 4 or 6, the supplier′s queue, one currency;

  - inside the issue: the `tpd_voucher_issue` lock (15 s), a check that the numbers are free, a conditional voucher_created change with a row-count check, and a count of the vouchers before finishing.
- **Conflicts [Deduced]:**
  - Request status is read before the lock and not re-read. A request rejected or cancelled meanwhile can still get a voucher; only voucher_created is protected.
  - Writers that do not take the number lock (X1) can take a number between the check and the write.
- **Partial failure** (MyISAM, so not atomic; the code records engine_atomic = 0):
  - The undo removes finished vouchers, their lines and their child batches, and sets the counter back.
  - The voucher being written when the failure happened is not in the undo list, so its rows stay.
  - Setting the counter back can undo another writer's number.
- **Safeguard: guard.** Take A2's REQ lock for every request, in id order, and re-read status 4 or 6 under it. Include the in-flight number in the undo, and set the counter back only if it has not moved since.
- **Staff:** management sees "try again" when a request is in use elsewhere.

R5. initiate

- **Reached from:** the desk pages, and from R6 at :283. The exact page lines need a read (F-1). **Internal calls:** none. **Method:** desk :1176 (target file).
- **Existing checks:**
  - the accountant privilege;
  - the voucher is neither cancelled nor rejected, and is not finalized;
  - no earlier initiation; the table has a unique key on the voucher;
  - a valid paying account, with a reason if the account changes;
  - batch vouchers only from R6; the separation check;
  - a payment date that is not in the future;
  - the amount equals the voucher's, or for batch vouchers is worked out from the rate.
- **Writes:** batch vouchers are re-rated (line amounts, then the voucher Amount where Aflag is 0). Then the initiation row is written. The voucher's bank is changed only `WHERE Aflag = 0`. Batch, audit and document rows follow.
- **Conflicts [Deduced]:**
  - If V1 authorises first, the bank change silently does nothing. The ledger credits the old bank while the initiation records the new one.

    - The re-rating checks Aflag once, then changes lines that V1 may be reading.
- **Partial failure:** amounts are re-rated but no initiation is recorded (they are re-derived on retry), or the initiation is recorded but the bank change fails.
- **Safeguard: guard.** Take AUTH, because initiation changes what authorisation will post. Re-read Aflag under it, refuse a needed bank change when the voucher is no longer pending, and check every write.
- **Staff:** the accountant may wait up to 10 seconds, and is told when a voucher was already authorised with another bank.

R6. bulk_initiate

- **Reached from:** tpd_bulk.php:158. **Internal call:** R5 once per issued voucher (:283). **Method:** desk :228 (target file).
- **Existing checks:** the accountant privilege; the batch exists; the lock `tpd_bulk_<id>` (20 s); it can be resumed; it stops at the first error.
- **Conflicts and partial failure:** as R5, voucher by voucher. A stopped run leaves the batch “initiation incomplete” and can be resumed.
- **Safeguard: guard.** Take AUTH once for the whole run, as A2 proposes for bulk Paid, then the batch lock.
- **Staff:** other authorisations wait, and are refused after 10 seconds, while a bulk initiation runs.

Further reads needed before approval

Code reads in the linked folder, read-only:

- **F-1** The pages that send these six requests: transport_advance.php near :784; booking_details.php near :2477; approved_expenses_report.php:521; payment_approval_v2.php:1476; the desk pages that call initiate; tpd_bulk.php:158.
- **F-2** DriverAdvancePaymentController::tour_transport and transfer_amount, which raise and authorise advances.
- **F-3** Every reader of consignee.transport_advance.
- **F-4** Batches::prepare and finish, PayDesk::isCancelled, Separation::check, Perms and the groups holding each privilege. Also reject_approved_expenses and the other writers of driver_expenses status.
- **F-5** Checksums of the files used here that F11 did not cover (AccessPolicyRoutes.php, Perms.php, Batches.php, InvoiceNumberModel.php, VoucherPaymentModel.php) against R0′s manifest.

Database read: M2 (section 1), for the unique index on voucher.Vnum and the table engines.

A gap in Stage One section 3

Stage One section 3 called the six routes the ones that raise, delete or re-point vouchers outside 4a. That list was not complete for routes that **raise pending vouchers**. Stage One's F5 output already showed more writers of `'Aflag' => '0'`. Their routes, from the route file, are below. **[Observed]** The Stage One report is left unchanged, as you asked; this paragraph records the gap.

Route (POST)	Method
accounts/voucher_payments/save	VoucherPaymentController::save (manual voucher entry)
accounts/creditor_invoice/complete_payment	CreditorInvoiceController::complete_payment (four writes)
dashboard2/transport/driver_advance/tour_transport	DriverAdvancePaymentController::tour_transport (pending driver advance)
dashboard2/transport/driver_settlement/setteled_payment	DriverPaymentSettlementController::complete_payment
dashboard2/hotel_management/hotel_payment/hotel_pay	NewDashboard/hotel/HotelPaymentController::hotel_payment
tour/booking_confirm/tour_transport and /setteled_payment	BookingConfirmController::tour_transport, ::complete_payment
not found by this search	UtilityBillPaymentController::createUtilityVoucher (private; called from two methods); NewCreditorInvoiceController::complete_payment123

None of these authorises a voucher. Their vouchers are authorised through V1, so C0.15 governs them once guarded. They do share X1 (voucher numbers) and X3 (lines tied by number), and hotel_pay is a second hotel-advance route like R2. Reading them as fully as R1–R6 is further read **F-6**.

4. The ledger's aflag and Bank_Date

Neither field marks a row as void in the application code. So, on code evidence, neither changes which rows count for R20–R23 or for balances. What the data actually holds in them is not established, and two SELECTs would show it.

Verified code behaviour. [Observed: code copy, backup copies of controllers excluded]

Field	Written by	Read by
aflag = 3	authorisation (V1, through acc_array, including the sup = 2 clearing journal); hotel voucher accept (V4); driver-advance transfer (V5, :476, :497); JournalEntryController (:113, :248, :276)	PettyCashReportController (:50, :101): the petty-cash report lists only aflag 3 rows
aflag = 1	DriverFundReturnController (:161, :183); TourDetailsSearchController advance_delete: “takes a tour advance out of the advance list” by setting every leg from 0 to 1, audited	TourDetailsSearchController: the tour advance list shows rows on 6-11-107-10381 with aflag 0, and advance_delete accepts only entries whose legs are all 0
aflag copied	ManualUpdateController :686 (bank_recon_record_insert, a retired tool that now refuses)	—
aflag not set	most other posters, for example expense acceptance (R3) and hotel accruals	—
Bank_Date	listed as an allowed field in AccountRunningModel and AccountRunning2024Model; **no code writes it**	**no code reads it**

voucher.bank_date is a different column, on the voucher table. Bank Payment Check and the desk’s Paid step set it.

What follows. [Deduced]

- aflag records which kind of routine posted a row. For tour advances it records whether the entry is still in the advance list.
- aflag 1 on an advance does **not** remove its rows from the ledger. Every reader other than the advance list still counts them.
- R0 and R20–R23 did not filter on aflag. Code evidence gives no reason they should have.
- Any Bank_Date values came from outside the current code: older versions, imports or direct edits.

Assumptions and open points.

- **[Assumption]** No row carries an aflag value written outside the code paths above, such as by direct edits.

- The column default, which applies when a poster does not set aflag, is not known. M1 shows it.
- Whether the accountant should treat an advance taken out of the list as reversed is an accounting judgement for Ravi's review, not a code question.

Checks proposed. Both are SELECTs, for your approval.

A1. How aflag and Bank_Date are spread, by ledger table and description category.

```
SELECT /*+ MAX_EXECUTION_TIME(60000) */ 'account_runing' AS ledger_table,
a.aflag,
       CASE WHEN UPPER(a.Discription) LIKE 'VOUCHER PAYMENT%' THEN 'voucher
payment'
            WHEN a.Discription LIKE 'Hotel Reservation%' THEN 'hotel
reservation accrual'
            WHEN a.Discription LIKE 'Lunch or Other Fee%' THEN 'lunch or
other fee'
            WHEN a.Discription LIKE 'Brought Forward Balance%' THEN 'brought
forward'
            WHEN a.Discription IS NULL OR a.Discription = '' THEN '(empty)'
            ELSE 'other (text withheld)' END AS description_type,
       COUNT(*) AS row_count, SUM(a.Dr) AS dr_total, SUM(a.Cr) AS cr_total,
       SUM(a.Bank_Date IS NOT NULL) AS rows_with_bank_date, MIN(a.Bank_Date)
AS first_bank_date, MAX(a.Bank_Date) AS last_bank_date
FROM avenir.account_runing a
GROUP BY a.aflag, description_type
UNION ALL
SELECT 'account_runing_2024', b.aflag,
       CASE WHEN UPPER(b.Discription) LIKE 'VOUCHER PAYMENT%' THEN 'voucher
payment'
            WHEN b.Discription LIKE 'Hotel Reservation%' THEN 'hotel
reservation accrual'
            WHEN b.Discription LIKE 'Lunch or Other Fee%' THEN 'lunch or
other fee'
            WHEN b.Discription LIKE 'Brought Forward Balance%' THEN 'brought
forward'
            WHEN b.Discription IS NULL OR b.Discription = '' THEN '(empty)'
            ELSE 'other (text withheld)' END AS description_type,
       COUNT(*), SUM(b.Dr), SUM(b.Cr),
       SUM(b.Bank_Date IS NOT NULL), MIN(b.Bank_Date), MAX(b.Bank_Date)
FROM avenir.account_runing_2024 b
GROUP BY b.aflag, description_type
ORDER BY 1, 2, 3;
```

A2. aflag and Bank_Date on the Stage One rows: the 8 unbalanced journals and the 30 future-dated rows. G3 in section 1 now returns both fields for the 38 sup = 0 vouchers' journals as well.

```
SELECT /*+ MAX_EXECUTION_TIME(60000) */ a.ID AS journal_id, a.L_Code, a.Dr,
a.Cr, a.Date, a.aflag, a.Bank_Date
FROM avenir.account_runing a
WHERE a.ID IN (24654, 24734, 25804, 25915, 25918, 27724, 27725, 28358)
   OR a.Date > '2026-12-31'
ORDER BY a.ID, a.Dr DESC, a.L_Code;
```

Expected 41 rows: 11 from R21 and 30 from R22.

How the results would be read.

- "Voucher payment" rows with aflag 3 and no Bank_Date match the code.
- Any other combination is reported as found, not explained away.
- If unusual values sit on the Stage One rows, those rows are flagged for Ravi's review. They are not dropped from any count.

5. Stage One evidence: inventory and a durable home

The evidence is in four places today. None of them is durable, and none is on your computer in a place you control. I propose one evidence folder outside the web server's folders, filled only after you approve it. **[Observed]** for the inventory; **[Proposal]** for the location.

Result files (this session's workspace)

The folder is stage1/ in the session scratch area. Full SHA-256 values:

File	Content	SHA-256
R20.1.txt	26 rows	398d0a71822352b45dfaa6c26c4ab1cc6e5057ff2c282bacc4241cd4fcb699b1
R20.2.txt	a note only: no result was captured	25b6b0b0468814e3ff7df4dd002e7d00d5765935e4f77e58775f9b74332eafbc
R20.3.txt	1 row (0)	de37b37de45d114784567b0c9eaadf22a09f58c7cda4eb2f6bab55e6a69e05f4
R21.1.txt	8 rows	4379d59e5fcca30c67b29382174888f8bb83f03fbd3183f804bcc81ed9677d4d

File	Content	SHA-256
R21.2.txt	11 rows	0c2af3edf99f106f6e73c18b692d90adea614225e40242af2f511e59d5baa79e
R22.1.txt	30 rows; free text in one column summarised in brackets	76e90f2be59ce98506dbbc6243e0671052808978cd57beb9611781e0b24fbbe7
R22.2.txt	30 rows, same treatment	bcec4b4bc43089f1d6db651772d6ef647395ac10f4b74bd88bcc4e192811a7cf
R23.1.txt	38 rows	27753a65f1e8e49f995b413310dde2341caf13609b8efc93945eef5559e8eec0
R23.2.txt	1 row	8f40d47f38876a67ab135c7a0d857a8909e95ab49b4774051f79b2e4d5fc3317
README.txt	what the files are	87c9661089b4f461ba95377738f01c889a3d658efb31230e35e03f87f625fbf7

Limits.

- The files were written from the parsed results, not from phpMyAdmin's raw responses.
- T1–T3 have no file; their values are in the Stage One report.
- The files disappear with this session.

Code-check outputs (the linked shell's scratch area on your computer, outside your folders)

File	Content	SHA-256 (start)
f3.txt	F3, 136 lines	b750afc81f8c3c8b
f4.txt	F4, 104 lines	1ceefc242ea50284
f7.txt	F7, 216 lines	8b919ff0e9ef7bd8
f9.txt	F9, 46 lines	3b550ea62ef8c452
f11_before.txt, f11_after.txt	44 checksums each, identical	3ff4aaaad4e79552 (both)
r30_vs_r0.txt	output of the check against R0's manifest: empty, meaning all matched	e3b0c44298fc1c14 (empty file)

File	Content	SHA-256 (start)
r0/final_files.sha256	R0's manifest of 2,487 files, 08:01 UTC	4983b936e77ac759

The outputs of F0, F1, F2, F5, F6, F8 and F10 were not saved to files. They exist only as printed output in this session. They can be re-saved word for word from that output, with no new reads.

Staged copies (this session's uploads area)

The target's TourPaymentDeskController.php (6a72efb8b8071ee9), Bulk.php (973feabc144732e6) and Vouchers.php (68eee3259bd54fc4) were staged at 20:27 UTC for the diff. The same area also holds older copies from earlier stages that are **not** Stage One evidence. One example is Mask.php from 11 October, whose checksum differs from the target's.

The uncaptured R20.2 response

[Observed]

Item	Value
Sent	20:32:25.514 UTC, through phpMyAdmin's own SQL handler, "show all rows"
Returned	21:08:26.292 UTC, after 2,160,766 ms
Response	HTTP 200; phpMyAdmin reported success; no result table and no message were found in it
Raw body	not kept: my capture parsed it and discarded the text. This was my omission.
Still available	the browser pane's network log for the phpMyAdmin tab holds the body (request 14036.8129)

Only its first part could be read through the tool: phpMyAdmin's usual page-settings block, which also carries a phpMyAdmin session token. The rest is cut off in the tool's output, so whether a result table follows is **not established**. The log lasts only while that tab stays open.

Possible cause [Assumption, not tested]. phpMyAdmin's default sign-in lifetime is 1,440 seconds (24 minutes), and R20.2 ran for 36. Whether that is why the result is missing is not established.

Proposed durable location

- **Where:** a new folder outside C:\wamp64\www, for example C:\AvenirEvidence\2026-10-14-stage1, or a folder you name. Anything under www can be served by Apache, so evidence must not go there. The application folder is not touched.
- **What goes in:**
  - the result files and README;
  - the code-check outputs and the R0 manifest;
  - the re-saved F0–F10 outputs;
  - the nine statement texts, with the SHA-256 values that matched A2;
  - PDF exports of the Stage One report and this package;
  - one SHA256SUMS file covering everything.
- **Kept apart:** the R20.2 body, only if you approve saving it, with the session token replaced by "[token removed]" before it is written.
- **What it needs from you:** folder access for that one path, granted on your computer. Nothing is written until you approve. The folder can then be made read-only and copied to your usual backup.

6. How approved statements would run

Statements run strictly one at a time. Each one's full response is saved before the next is sent, and any failure stops the run. **[Proposal]** These are rules for the run you approve; nothing has been sent.

1. **Before starting.** You choose a time when the local app is idle and sign in to phpMyAdmin yourself. I confirm the target with T1, if you include it in the approved list, and stop on any difference.
2. **Order.**
   a. T1, M1, M2.
   b. E1, only if you approve it.
   c. G1, G2, G3, G4.
   d. A1, A2.
   e. R20.2r.
   - The quick metadata reads come first, and the statement with the longest locks comes last.

3. **Gate on each statement.** The text must be exactly the approved text: its SHA-256 matches the list in section 7. It must be a single statement beginning with SELECT, or EXPLAIN for E1 only.
4. **One at a time.**
   - A statement is sent only after the previous response has fully arrived and been saved.
   - No other tab sends queries meanwhile.
   - Start and finish times are recorded to the millisecond.
5. **Capture first, then read.**
   - The raw response body is saved before it is parsed, with phpMyAdmin's session token replaced by "[token removed]". Its SHA-256 is recorded.
   - The parsed rows are then compared with phpMyAdmin's own "Showing rows … total".
6. **Hint check.** Before R20.2r, M1's saved response must show that phpMyAdmin sent the hint unchanged. If it did not, the run stops before R20.2r.
7. **Stop rules.** The run stops and is reported, with no retry and no rewording, on any of these:
   - an error, including error 3024 (time cap reached);
   - a missing result table;
   - a row count different from phpMyAdmin's total;
   - a failed built-in check (G2 totals, the R20.2r monthly match);
   - a sign-in prompt.
8. **Afterwards.** Results go to the evidence folder you approve (section 5), with a SHA256SUMS file, followed by one report. None of the statements has been executed anywhere, including in the cloud. Their syntax was checked by reading only.

7. Decisions and approvals needed

The investigation stays open, and nothing proceeds to installation. These are the decisions that would let the next checks run. **[Proposal]**

1. Statement texts. Approve each by its SHA-256, computed over the exact text in this package.

Id	Purpose	SHA-256 of the text
T1	target confirmation, unchanged from Stage One	ea630a9d40b12b39a1ef0a113b2743e526356f09e859b81c4ded4baf1180e37d

Id	Purpose	SHA-256 of the text
M1	columns, types, defaults	534af761d2de9c25ffff7e25ad5f1b51c2ab192b8bad36fd91b18ad8f68a05cf
M2	sizes, engines, indexes	a0dcb5261c0c5569b9d2241587ed18dce955710609d44f592980b4fc0e3c73bd
E1	optional query plan: `EXPLAIN FORMAT=TREE` followed by the R20.2r text	90cae3afef5e0bc05570f73ec7d5c1ca1eb2f983e9693c41fb83f44ef388de81
G1	section 8 comparison, identical to the Stage One text	333b765bfc4253a2421c1dc2d901cc18dc598d13604dcfb6604ac713c4aa1e6c
G2	per-voucher reconciliation	7b7496b4218fd35d4163e8326b69cea142586ca2a0ee86e666ab2dadefdacc0c
G3	journal legs on 1-6-8-9750	095226d60b7d1898a30d96f82652124b688e997e18e5f5ffe9900bd98e5e39cd
G4	voucher lines	3f11a21874d16511f3f3a94c4ef348dd5d3ab3134c52af7231a6f6ac1c5a3c45
A1	aflag and Bank_Date spread	d2a40f46fb36c160b5c8b57a5972a1f3fd6504162bf8ec8a755f70dbb7c33f42
A2	aflag and Bank_Date on Stage One rows	e886caac95659894049359fb707b07f8aae261876b62029edebf49f63d192dc7
R20.2r	revised R20.2	48813607fb2a688fbb5dde86e406d256861d9dde199e853f932f8b0186b34169

2. E1. Include the query-plan read (not a SELECT), or run R20.2r with only its 2-minute cap.

3. Run window. When the local app may be locked for reading for up to 2 minutes by R20.2r, and briefly by the others.

4. Containment (section 3). This is for your operations call. Nothing is designed further without it.

- R1 delete_advance: refuse (recommended) or guard.
- R2 hotel_payment: refuse (recommended) or guard.
- R3 to R6: guards as described.
- Whether to add the voucher-number lock (X1) for every voucher writer, including the routes in the Stage One gap table.

5. Further code reads F-1 to F-6. These are read-only inspection in the linked folder, needed before any guard can be specified.

6. Evidence.

- Choose a folder outside C:\wamp64\www and grant access to it.
- Decide whether to save the R20.2 response body from the browser log, with the token removed, before that tab closes.

Unchanged. Ravi is not contacted and no records are sent. A0, D14 and containment implementation stay pending, and staff and access decisions stay with you. The Stage One report, A1, A2, r3.3–r3.5 and the Security proposal have not been edited.